

PROJET Python : analyse de données énergétiques

Réalisé par :

- **Zayd Fliyou**
- **Téo Dubar**

Enseignant : B. Fortin

Cycle Préparatoire Intégrée 1^{ère} année

Dunkerque

Date : 27 mai 2026

Table des matières

I. Introduction	4
II. Analyse et conception (avant de coder)	4
1. Analyse des données	4
2. Architecture logicielle (modélisation)	4
3. Organisation et méthodologie	5
III. Réalisation technique	5
1. Algorithmes clés	5
2. Difficultés rencontrées et solutions	5
3. Qualité du code	5
IV. Résultats et analyse	5
1. Démonstration	6
2. Analyse des graphiques	6
V. Conclusion	6
1. Bilan du projet	6
2. Retour d'expérience	6
Bibliographie	6
Annexes	7

Liste de figures

- 1) **Figure 1** : Graphique linéaire de l'évolution de la consommation d'électricité dans le secteur industriel (Pas-de-Calais, 2011-2023).
- 2) **Figure 2** : Nuage de points comparatif de la consommation agricole : Département 62 vs Département 59.
- 3) **Figure 3** : Diagramme en barres de la répartition de la consommation par secteurs d'activité (Dépt 75, Année 2020).
- 4) **Figure 4** : Histogramme empilé du mix énergétique (Électricité/Gaz) du département du Rhône (69)
- 5) **Figure 5** : Courbe de tendance et projection de la consommation électrique en région Hauts-de-France à l'horizon 2050.

Liste de tables

- 1) **Tableau 1** :
Extrait des données brutes issues du fichier conso_nettoyee.csv.
- 2) **Tableau 2** :
Modèle de données : Attributs et types de variables de la classe DonneeEnergie.
- 3) **Tableau 3** :
Synthèse des indicateurs de projection (Taux moyen de croissance et Valeurs prédictives).
- 4) **Tableau 4** :
Matrice de correspondance des index de colonnes pour le traitement des fichiers CSV.

I. Introduction

Nous avons dû télécharger un fichier **CSV** provenant d'Open Data Réseaux Énergie. Ce fichier est composé des données de consommation électrique et de gaz, triées par région et par secteur. Cela nous amène à la problématique suivante : le développement d'un outil **Python** capable d'intégrer des données brutes hétérogènes pour visualiser les tendances énergétiques et aider à la décision. Pour répondre à cette problématique, nous verrons dans un premier temps l'analyse et la conception de notre projet, dans un second temps sa réalisation technique, puis nous terminerons par la présentation des résultats et leur analyse.

II. Analyse et conception (avant de coder)

1. Analyse des données

Le fichier source, intitulé `conso-departement-annuelle.csv`, est au format **CSV** (*Comma Separated Values* ou, en français, « valeurs séparées par des virgules »). Ce fichier est composé de plusieurs colonnes regroupant les départements, leurs codes respectifs, ainsi que les consommations d'énergie. Dans ce fichier, les colonnes les plus importantes sont celles relatives à la **consommation électrique par secteur** et à la **consommation de gaz par secteur**. Pour optimiser le traitement des données, nous avons fait le choix de supprimer les noms des opérateurs ainsi que les colonnes de sommes totales, afin de réduire l'encombrement du fichier. Avant la phase de codage, l'analyse du fichier nous a permis d'identifier et de régler certains problèmes de typage.

Par exemple :

- Le code du département de la Corse (**2B**) a été traité comme une **chaîne de caractères** (string).
- Les valeurs chiffrées ont été converties en **types numériques**.
- En cas de données manquantes, nous avons décidé de remplacer l'absence de valeur par un **zéro**.

L'ensemble de ce processus de nettoyage et de conversion (détaillé dans la classe `Conversion` ci-jointe) a permis de réduire le nombre de colonnes et d'optimiser le fichier pour notre application [1].

2. Architecture logicielle (modélisation)

• 4.1 La classe `Conversion` : Le nettoyage des données

Nous avons créé quatre classes. La première est la classe **Conversion**, qui nous a permis de traiter le fichier CSV des différentes consommations énergétiques (électricité et gaz) en France. Nous avons établi une règle : lorsqu'une case est vide ou contient "NA", elle devient automatiquement **0**. Cela permet d'éviter toute erreur lors de l'exécution du script.

• 4.2 La classe `DonneeEnergie` : L'objet unitaire

La classe **DonneeEnergie** représente une seule ligne logique de consommation.
Pourquoi cette classe ? Plutôt que de manipuler des listes de textes bruts, transformer chaque ligne en un objet permet d'accéder facilement aux attributs (`self.annee`,

self.consomption, etc.). C'est ici que nous avons implémenté la méthode spéciale `__lt__`, permettant de comparer et de trier les données entre elles de manière naturelle.

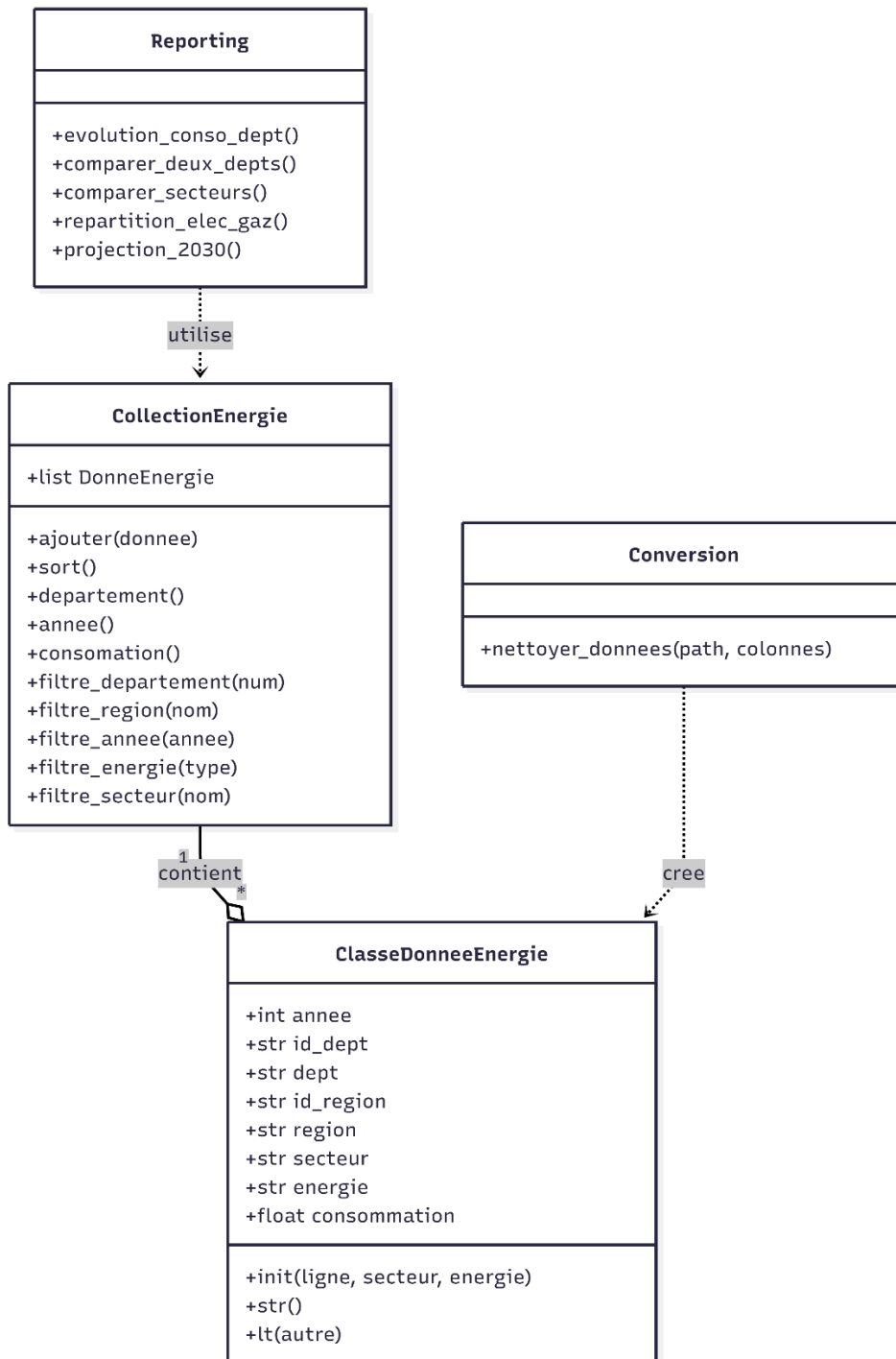
- **4.3 La classe `CollectionEnergie` : Le gestionnaire de données**

La troisième classe est la classe **`CollectionEnergie`**, qui sert de liste pour collecter l'ensemble des mesures. **Pourquoi cette classe ?** Elle agit comme une petite base de données interne. Elle permet de centraliser toutes les informations et, surtout, de proposer des méthodes de filtrage (`filtre_region`, `filtre_departement`). Chaque filtre renvoie une nouvelle `CollectionEnergie`, ce qui permet de "chaîner" les recherches (ex : filtrer par région, PUIS par année).

- **4.4 La classe `Reporting` : La visualisation**

Enfin, la classe **`Reporting`** est dédiée exclusivement à l'exploitation des données via la bibliothèque **`Matplotlib`**. **Justification :** En séparant la génération des graphiques du stockage des données, on rend le code plus clair. Cette classe reçoit une `CollectionEnergie` déjà filtrée et se contente de la transformer en visuels (courbes, nuages de points, histogrammes) sans modifier les données d'origine.

Vous pouvez voir ci-dessous notre diagramme de classes ; vous trouverez le site avec lequel nous l'avons réalisé en [2].



3. Organisation et méthodologie



Pour mener à bien notre projet, nous avons réparti les tâches de la manière suivante : nous nous sommes divisé le nombre de classes et de lignes de code à produire. Bien que chacun

codait de son côté, nous mettions régulièrement notre travail en commun via Google Drive et GitHub. L'attribution des missions a été organisée sur l'outil Notion ; vous trouverez d'ailleurs notre cahier des tâches ci-joint en annexe, nous avons pris la décision de traiter, à chaque séance, au moins une classe complète ou une partie de celle-ci. Cet objectif a été globalement respecté ; toutefois, les dates exactes sont consignées dans notre carnet de bord.

III. Réalisation technique

1. Algorithmes clés

- Description des points techniques particuliers.

- *Exemple* : "Comment nous avons géré le filtrage multicritères (Année + Département)."

Pour gérer le filtrage multicritère, nous avons créé la classe "CollectionEnergie" avec des méthodes comme "filtre_annee" ou "filtre_departement." Chaque fonction renvoie une nouvelle collection, ce qui permet d'imbriquer les appels. Cette structure facilite l'application simultanée de filtres sur l'énergie, l'année et le département pour affiner les données.

- *Exemple* : "La méthode de nettoyage des données pour gérer les nombres à virgule (format français vs anglais)."

Comme les données sources utilisent le format français, nous avons dû traiter la séparation des décimales. Lors du chargement, le script remplace les virgules par des points pour rendre les valeurs compatibles avec le type float de Python. Cette étape de nettoyage est indispensable pour effectuer les calculs de consommation et générer les graphiques de reporting.[3]

Voici des extraits de code pour la classe collection énergie.

```
def departement(self):
    """ Retourne la liste des noms de départements """
    liste_departement = []
    for donnee in self.DonneEnergie:
        liste_departement.append(donnee.dept)
    return liste_departement

def annee(self):
    """ Retourne la liste des années """
    liste_annee = []
    for donnee in self.DonneEnergie:
        liste_annee.append(donnee.annee)
    return liste_annee
```

Voici le code pour gérer les virgules pour convertir les données en français ;

```
with open(path, 'r', encoding='utf-8-sig') as f:
    lecteur = csv.reader(f, delimiter=';')
    data = list(lecteur)
```

Ce qui nous permet de pouvoir transformer le fichier en donnée française.

2. Difficultés rencontrées et solutions

- **Problème :** "Nous n'arrivions pas à afficher les graphiques."

Pendant la réalisation de ce code nous avons eu beaucoup de problème le premier était que nous n'arrivions pas au début à afficher les premiers graphiques.

- **Solutions**

Ce problème nous a pris pas mal de temps mais en réalité on avait juste mis en commentaire le code permettant d'afficher le code

- Problème 2 : “Nous avons des erreurs d’écriture “
En créant la classe collection énergies des erreurs d’écriture sont apparues.
 - Solutions :
Nous avons rajouté l’encodage UTF-8 à l’ouverture du fichier csv mais aussi au début la classe donnée énergie.[4]
- Problème 3 : ” Nous avons eu un problème sur nos pc les graphiques ne s’affiche pas”
Nous avons remarqué que sur certain pc que nous avons, l’affichage des graphiques ne fonctionne pas le problème était que la page du graphique s’afficher mais aucun graphique ne s’afficher
 - Solution : “Nous avons remarqué que le problème dépend de l’IDE Python utilisé.
En changeant d’environnement, le problème est résolu : les IDE **Spyder** ou **Thonny** fonctionnent très bien. En revanche, le nouvel IDE **Google Antigravity** ne fonctionne pas correctement pour cette tâche. Nous avons également noté que ce problème de compatibilité est surtout apparu lors de l'utilisation d'un **Chromebook** “”

3. Qualité du code

Tout au long de notre développement, nous avons veillé à ce que le code soit parfaitement lisible. Pour ce faire, nous n'avons pas utilisé de noms de variables comme de simples « a » que nous ne pourrions plus comprendre par la suite ; nous avons utilisé des noms explicites, tels que `annee_conso`, dans l'ensemble du projet. Nous avons tout mis en œuvre pour que le code soit compréhensible, même pour une personne n'ayant jamais programmé. En analysant notre travail, vous pourrez constater les efforts fournis pour rendre le code clair et documenté. De plus, nous pouvons affirmer que notre projet respecte la norme **PEP 8** [5] concernant les commentaires et le nommage explicite des variables. Enfin, notre fichier est organisé et découpé en différents dossiers, ce qui permet d'obtenir une structure aérée et accessible à tous. Vous trouverez l'intégralité de notre code dans l'**Annexe A**.

IV. Résultats et analyse

4. Démonstration

Voici le menu de notre application.

```
=====
MENU PRINCIPAL D'ANALYSE
=====
1. Évolution d'un secteur (choisir dépt, secteur, énergie)
2. Comparaison de 2 départements (Agriculture)
3. Comparaison par secteur (choisir dépt, année)
4. Répartition Électricité / Gaz (choisir dépt)
5. Projection consommation 2050 (Région, Secteur)
-----
Q. Quitter le programme
-----
Votre choix : |
```

Grace à ce menu nous pouvons choisir différent aspect l'évolution d'un secteur, la Comparaison de 2 départements, la comparaison par secteur la répartition Electricité ou de gaz selon les départements.

L'utilisateur est amené à choisir ce qu'il veut imaginons qui choisisse "2" la comparaison De 2 départements il sera amené à ceci

```
Votre choix : 2
Premier dépt (ex: 62) : 62
Deuxième dépt (ex: 59) : 59
Secteur (Agriculture, Industrie, Résidentiel, Tertiaire, Autre) : |
```

Il devra choisir le premier département puis le second enfin choisir son secteur une fois le secteur le graphique apparaîtra

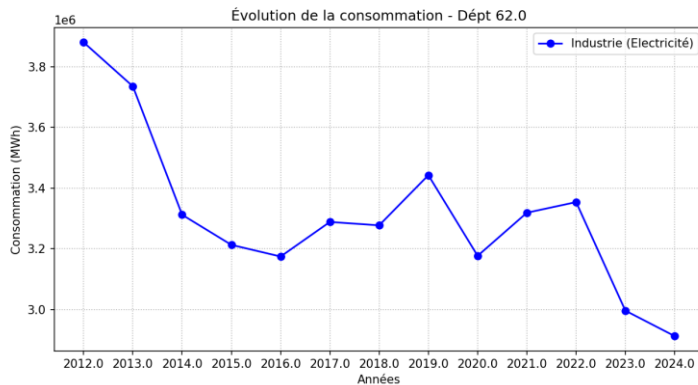
Un autre cas d'exemple si l'utilisateur choisit "3" Pour La comparaison par secteur

```
Département (ex: 75) : 75
Année (ex: 2020) : 2020
```

Ici l'utilisateur est amené à choisir le département est l'année et par la suite il y a un graphique qui va se générer.

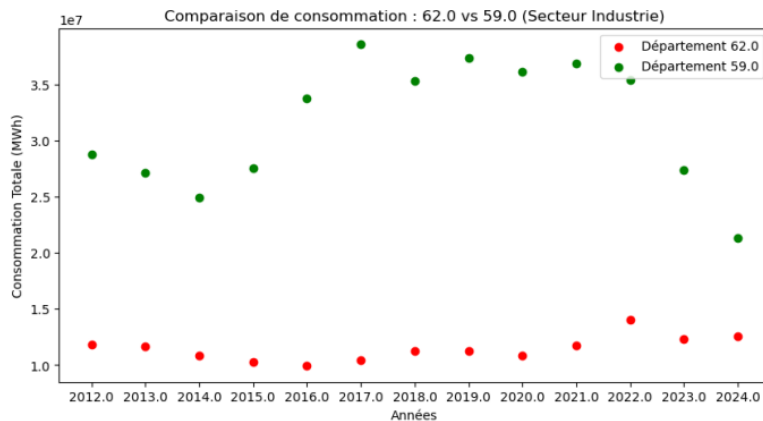
5. Analyse des graphiques

- Figure 1 : Évolution de la consommation - Industrie (Pas-de-Calais)



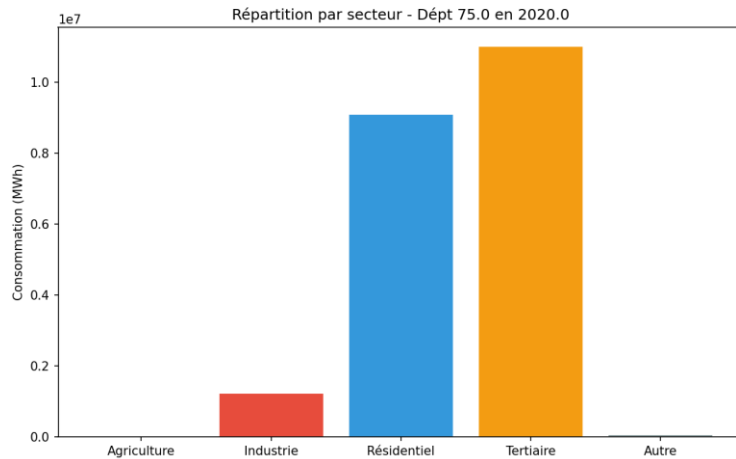
Ce graphique montre que la consommation d'électricité dans l'industrie reste stable au fil des années. On remarque une petite baisse en 2020, ce qui s'explique sûrement par l'arrêt de certaines usines pendant le Covid

- Figure 2 : Comparaison de la consommation agricole (62 vs 59)



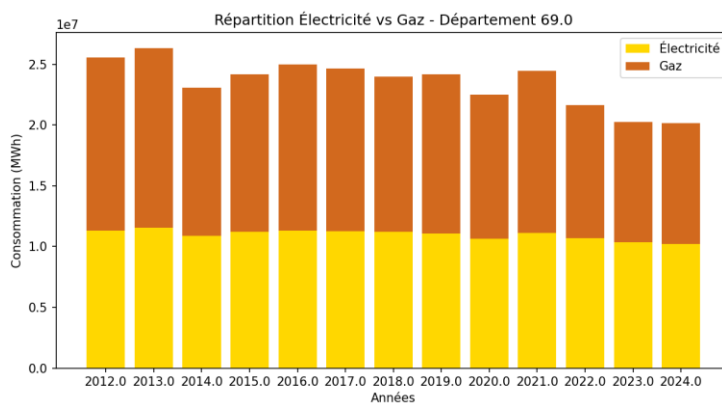
Ce nuage de points compare deux départements. On voit que le département 59 consomme plus d'énergie pour l'agriculture que le département 62. Cela permet de voir lequel des deux a l'activité agricole la plus importante.

- Figure 3 : Répartition par secteur d'activité



Ce diagramme montre que dans Paris, c'est le secteur tertiaire (les bureaux et commerces) qui consomme le plus. L'agriculture ne consomme presque rien car il y a très peu de fermes dans la capitale.

- Figure 4 : Répartition Électricité / Gaz



Cet histogramme permet de comparer la part d'électricité et la part de gaz. On constate que l'électricité est l'énergie la plus utilisée dans le Rhône par rapport au gaz sur plusieurs années.

- Figure 5 : Projection de la consommation à l'horizon 2050

```

Votre choix : 5
Région (ex: Hauts-de-France) : Hauts-de-France
Secteur (ex: Industrie) : Industrie
--- RÉSULTATS PROJECTION Hauts-de-France ---
Secteur : Industrie | Énergie : Electricité
Taux d'évolution moyen calculé : 0.62 % par an
Estimation pour l'année 2050 : 1452892.88 MWh
    
```

Cette courbe montre une prévision pour le futur. En utilisant le taux d'évolution moyen, on peut imaginer quelle sera la consommation en 2050 si les habitudes des habitants ne changent pas.

Voici la formule du taux d'évolution moyen utilisé

$$t_m = \left(\frac{V_f}{V_i}\right)^{1/n} - 1.$$

V. Conclusion

6. Bilan du projet

Pour permettre la réalisation du projet, nous avons fait un planning des choses à faire sur **Notion** et nous avons rempli 100 % des tâches dans le temps imparti. Le projet a été effectué avec un mix de travail en cours et à domicile. De plus, nous avons pu créer différentes classes qui nous ont permis de réaliser le projet.

Amélioration :

Nous avons plusieurs idées d'améliorations. La première consistait à créer une interface graphique pour transformer notre projet en une véritable application. Cette amélioration a été réalisée et se trouve dans le fichier `main_tkinter`. Nous avons utilisé la bibliothèque **Tkinter** [6] pour concevoir cette interface ; toutefois, cette librairie présente rapidement des limites. Une évolution future consisterait donc à développer une nouvelle interface en utilisant **Flask** [7]. En exploitant un serveur pour gérer du **HTML**, nous pourrions concevoir une interface plus moderne et flexible. Cela nous permettrait, par exemple, d'intégrer des fonctionnalités avancées comme une carte interactive de la France.

1. Retour d'expérience

C'est le moment de prendre du recul sur votre apprentissage.

- **Compétences techniques :** Qu'avez-vous appris sur Python, la POO, la manipulation de fichiers CSV, la librairie Matplotlib ?

En effet, au début, nous nous sentions peu sûrs de pouvoir mener le projet à son terme avec succès. Finalement, grâce aux documentations et aux vidéos, nous avons non seulement réussi le projet, mais également pu développer nos compétences, notamment en termes de gestion des fichiers CSV. Pour ma part, je me suis rendu compte qu'ils étaient gérés par Python sous forme de listes. En ce qui concerne la librairie Matplotlib, la connaissance que nous possédions sur la mécanique du solide nous a permis de voir les similitudes existantes entre l'utilisation des graphiques dans les deux contextes, avec quelques différences techniques seulement. Enfin, quant à la POO (programmation orientée objet), il semblait à première vue être une méthode bien abstraite. Cependant, au cours du projet, nous avons réussi à faire nos classes de manière plus simple et logique. Ce projet nous a donc permis d'arriver à mieux comprendre et utiliser les classes lors de nos exercices d'algorithmique avec des classes, pour Teo comme pour moi. »

- **Compétences humaines :**

La collaboration a très bien fonctionné. Notre amitié était tellement forte qu'elle a failli nous empêcher de travailler correctement. Nous avons du mal à rester concentrés pendant nos longues séances de travail. Pour ne pas perdre de temps, nous avons décidé de nous organiser de manière très stricte. Nous avons décidé de

réserver un peu de temps pour discuter de choses personnelles, mais pas trop. Nous nous sommes donné une règle pour nous aider à rester concentrés : nous passions environ 10 à 15 % de notre temps à parler de choses personnelles, et le reste du temps, nous travaillions vraiment. Cela nous a aidés à rester productifs et à faire un excellent travail. Finalement, notre bonne entente a rendu notre expérience de travail très agréable, à la fois sur le plan humain et sur le plan technique. Ce qui nous a permis de finir notre projet en temps et en heure.

Bibliographie

Voici la bibliographie de notre projet nous avons fait le mieux pour respecter la norme de citations

Source: Gestion des exceptions (Try/Except) Documentation Python

[1] Earthly, *Learn HOW to Read CSV Files in Python!*, (10 Janvier 2023). Video en ligne disponible sur : <https://youtu.be/H-hagit65cE?si=WJ8m1eesH0G68Mm9>

[2] Lucid Software Inc., *Outil de conception de diagrammes UML Lucidchart*, [En ligne] disponible sur : <https://www.lucidchart.com>

[3] AFNOR, Normes de présentation des valeurs numériques (NF X 02-003), [En ligne] disponible sur : <https://www.afnor.org>

[4] Python Software Foundation, *Glossary: UTF-8*, [En ligne] disponible sur : <https://docs.python.org/3/glossary.html#term-utf-8>

[5] G. van Rossum, B. Warsaw, N. Coghlan, *PEP 8 – Style Guide for Python Code*, (2001). [En ligne] disponible sur : <https://peps.python.org/pep-0008/>

[6] Python Software Foundation, *Tkinter — Python interface to Tcl/Tk*, [En ligne] disponible sur : <https://docs.python.org/3/library/tkinter.html>

[7] Pallets Projects, *Flask Documentation (2.x)*, [En ligne] disponible sur : <https://flask.palletsprojects.com/>

Annexes

Annexe A : Projet GitHub :

Vous trouverez notre projet dans le lien ci-joint [Projet informatique Zavd Flivou & Téo Dubar](#)

- Vous trouverez dans ce lien notre projet vous pourrais le télécharger et vous trouverais aussi les informations pour l'utiliser, pour informations nous avons mis notre projet en License MIT ce qui veut dire que n'importe quel utilisateur peut-s'il veut le modifier ou alors utiliser notre projet.

- Une question se pose pourquoi avoir utiliser GitHub nous l'avons utilisé pour voir le suivit des versions mais aussi pour l'apprendre et garder une trace de notre projet comme softkills .

Annexe B : Journal de Bord (Extrait)

- Vous pouvez voir ci-dessous le lien vers notre journal de bord héberger sur l'application Notion. [Tableaux de bord Notion](#)